Cardmarket Expansion List Extractor - Complete Documentation (TCG-Only)

Table of Contents

Overview

The **Cardmarket Expansion List Extractor** is a Python utility designed to scrape and extract a complete list of **TCG (Trading Card Game)** Yu-Gi-Oh! expansions from Cardmarket.com, while filtering out OCG (Official Card Game) expansions. This lightweight script serves as a prerequisite for the main card scraper, providing the expansion metadata needed to iterate through all available TCG card sets.

Purpose

- Extract expansion IDs and names from Cardmarket's search filter dropdown
- **Filter out OCG expansions** (Japanese/Asian market cards)
- **Keep only TCG expansions** (Western market cards)
- Generate a master list of TCG-only Yu-Gi-Oh! expansions
- Provide clean, duplicate-free data for downstream processing
- Enable automated expansion discovery without manual data entry

Key Features

- **TCG-only filtering**: Automatically excludes OCG expansions from output
- **Single-purpose design**: Focused solely on expansion list extraction
- **CloudScraper integration**: Bypasses anti-bot protection automatically
- **Duplicate prevention**: Ensures each expansion ID appears only once
- **HTML entity decoding**: Properly handles special characters (`&` → `&`)
- **Validation checks**: Verifies data integrity before saving
- **Simple execution**: No configuration required, runs out-of-the-box
- **Informative output**: Shows TCG/OCG statistics and excluded expansions

Output

The script produces `cardmarket_expansion_list.json` containing **TCG expansions only**:

```
[
  {
    "expansion_id": 1651,
    "expansion_name": "2-Player Starter Deck Yuya & Declan"
  },
  {
    "expansion_id": 5420,
    "expansion_name": "2-Player Starter Set"
  }
]
```

OCG expansions are excluded from the output file but reported in console output.

Prerequisites

Required Python Libraries

```
import json
import cloudscraper
from bs4 import BeautifulSoup
import time
import re
```

Installation

```
pip install cloudscraper beautifulsoup4
```

System Requirements

- Python 3.7 or higher
- Internet connection
- ~5-10 MB disk space for output file

Estimated Runtime

- **Total execution time**: 5-10 seconds
- **Network requests**: 2 (warmup + main request)
- **Data size**: ~130-140 KB HTML download

Input/Output Files

No Input Files Required

This script operates standalone without requiring any input files. It directly queries Cardmarket.com for the expansion list.

Output File

`cardmarket_expansion_list.json`

- Contains **TCG-only** Yu-Gi-Oh! expansions from Cardmarket
- **OCG expansions are excluded** from this file
- Structure:

```
[
  {
    "expansion_id": 1234,
    "expansion_name": "Example TCG Set Name"
  }
]
```

Field descriptions:

- `expansion_id` (integer): Unique identifier for the expansion on Cardmarket
- `expansion_name` (string): Human-readable name of the expansion

Data characteristics:

- **Expected count**: ~900-1000 unique TCG expansions (varies by market)
- **Excluded count**: ~200-300 OCG expansions (filtered out)
- **Encoding**: UTF-8 (supports international characters)
- **Format**: JSON with 2-space indentation

- **Sorting**: Maintains order from Cardmarket dropdown

Debug File (Optional)

`debug.html` (only created if extraction fails)

- Contains the full HTML of the fetched page
- Used for troubleshooting extraction issues
- Allows manual inspection of page structure

Core Components

Configuration Constants

```
BASE_URL = "https://www.cardmarket.com"
SEARCH_URL = f"{BASE_URL}/en/YuGiOh/Products/Search?searchMode=v1&idCategory=0&idExpansio
OUTPUT_FILE = 'cardmarket_expansion_list.json'
USER_AGENT = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36...'
```

URL Parameters:

- `searchMode=v1`: Use search version 1
- `idCategory=0`: All categories (defaults to showing filters)
- `idExpansion=0`: All expansions (shows full dropdown)
- `onlyAvailable=on`: Only products with active offers
- `idRarity=0`: All rarities
- `perSite=1`: Results per page (minimal for faster load)

Functions

`create_scraper()`

Creates and configures a CloudScraper instance with realistic browser headers.

Purpose:

- Initialize web scraper with anti-detection capabilities
- Set realistic User-Agent and browser headers
- Configure connection parameters

Returns: CloudScraper instance ready for requests

Headers configured:

- User-Agent (Chrome on Windows)

- Accept types (HTML, XHTML, XML)
- Accept-Language (English, German)
- Accept-Encoding (gzip, deflate, br)
- DNT (Do Not Track flag)
- Sec-Fetch-* (modern browser security headers)
- Referer (navigation from homepage)

Anti-detection features:

- CloudScraper automatically solves JavaScript challenges
- Realistic header combination mimics genuine browser
- Connection keep-alive for session persistence

`is_ocg_expansion(expansion_name)`

NEW FUNCTION: Determines if an expansion is OCG based on its name.

Parameters:

- `expansion_name` (string): The name of the expansion

Returns:

- `True` if expansion contains "OCG" (uppercase)
- `False` if expansion is TCG (no "OCG" in name)

Detection logic:

```
return 'OCG' in expansion_name
```

Examples:

- `"Legend of Blue Eyes White Dragon (OCG)"` → **True** (excluded)
- `"Structure Deck OCG Edition"` → **True** (excluded)
- `"Legendary Collection"` → **False** (included)
- `"OCG Special Pack"` → **True** (excluded)

Case sensitivity:

- Only detects uppercase "OCG"
- "ocg" or "Ocg" would NOT be detected (unlikely to appear)
- Ensures accurate filtering based on Cardmarket's naming convention

`extract_expansions(html)`

Parses HTML to extract all expansion options from the dropdown menu, **separating TCG and OCG**.

Parameters:

- `html` (string): Full HTML content of the page

Returns:

- `tcg_expansions` (list): TCG expansion dictionaries
- `ocg_expansions` (list): OCG expansion dictionaries (excluded from output)

Extraction process:

1. Parse HTML with BeautifulSoup
2. Find `<select>` element with `name="idExpansion"`
3. Extract all `<option>` elements (including those in `<optgroup>` tags)
4. Filter out invalid entries (empty, value="0", non-numeric)
5. Decode HTML entities
6. **Check if OCG or TCG** using `is_ocg_expansion()`
7. Store in separate TCG/OCG dictionaries to prevent duplicates
8. Convert to lists and return both

Duplicate handling:

- Uses dictionary with expansion_id as key
- First occurrence of each ID is kept (per TCG/OCG category)
- Subsequent duplicates are skipped
- Reports count of removed duplicates

Data validation:

- Skips options with empty values
- Skips "All" option (value="0")
- Only accepts numeric expansion IDs
- Validates each entry before adding

`main()`

Main execution flow coordinating all operations.

Execution sequence:

1. Display header and version
2. Create scraper instance

3. Warm up session (visit homepage)
4. Fetch search page with expansion dropdown
5. Extract expansions from HTML (separate TCG/OCG)
6. **Display TCG and OCG statistics**
7. **Show excluded OCG expansions** (first 5)
8. Validate for duplicates
9. Display preview of TCG expansions (first/last 5)
10. **Save only TCG expansions to JSON file**
11. Display completion status

Error handling:

- HTTP errors (non-200 status codes)
- Connection timeouts
- HTML parsing failures
- File write errors
- Keyboard interruption (Ctrl+C)

Output verbosity:

- Progress messages for each step
- Success/failure indicators (✓/✗)
- Statistics (total options, TCG count, OCG count, duplicates removed)
- **Excluded OCG expansions list** (first 5 examples)
- Preview of TCG results
- Confirmation of file save

Extraction Logic and Concepts

Target URL Analysis

Search URL structure:

```
https://www.cardmarket.com/en/YuGiOh/Products/Search?
  searchMode=v1
  &idCategory=0
  &idExpansion=0
  &onlyAvailable=on
  &idRarity=0
  &perSite=1
```

Why this URL?

- The search page contains filter dropdowns
- `idExpansion=0` ensures the expansion dropdown is populated with all options
- The dropdown lists all available expansions for filtering
- This is the canonical source for expansion metadata on Cardmarket

HTML Structure

Dropdown location:

```
<select name="idExpansion" id="idExpansion...">
  <option value="0">All</option>
  <option value="1651">2-Player Starter Deck Yuya &amp;amp; Declan</option>
  <option value="4665">Yu-Gi-Oh! The Dark Side of Dimensions Movie Pack (OCG)</option>
  ...
</select>
```

Key characteristics:

- `<select>` element has `name="idExpansion"` attribute
- Each expansion is an `<option>` with numeric `value` attribute
- Option text is the expansion name
- May contain `<optgroup>` tags for categorization
- HTML entities are encoded (`&amp;` instead of `&`)
- **OCG expansions contain "OCG" in their name**

BeautifulSoup Parsing Strategy

Finding the dropdown:

```
select_element = soup.find('select', attrs={'name': 'idExpansion'})
```

- Searches for `<select>` tag with specific name attribute
- Direct, unambiguous identification
- Robust to page structure changes

Extracting all options:

```
options = select_element.find_all('option')
```

- Recursive search (default) finds options at any depth
- Automatically handles `<optgroup>` nesting
- Returns all `<option>` elements regardless of hierarchy

Why this works:

- BeautifulSoup's `find_all()` is recursive by default
- Finds nested elements inside `<optgroup>` tags
- No need to explicitly handle optgroup structure
- Simple and maintainable

Data Cleaning

Filtering process:

1. **Empty values**: `if not value` - Skip blank options
2. **"All" option**: `if value == '0'` - Skip filter reset option
3. **Non-numeric**: `if not value.isdigit()` - Only accept expansion IDs
4. **Type conversion**: `int(value)` - Convert string to integer
5. **HTML entities**: `text.replace('&amp;', '&')` - Decode entities
6. **OCG detection**: `is_ocg_expansion(name)` - Separate TCG from OCG

Why each filter matters:

- Empty values are placeholders or errors
- "All" is not an actual expansion
- Non-numeric values are invalid expansion IDs
- Integer IDs are needed for API/URL construction
- Proper decoding ensures readable names
- **TCG/OCG separation ensures correct output**

Session Warmup

Purpose:

```
scraper.get(f"{BASE_URL}/en/YuGiOh", timeout=15)
time.sleep(2)
```

Why warm up?

- Establishes legitimate browsing session
- Loads site cookies and JavaScript state
- Reduces suspicion from anti-bot systems
- Mimics human navigation pattern

What it does:

1. Visits Cardmarket homepage
2. Receives cookies and session tokens

3. Establishes connection state
4. Waits 2 seconds (human-like delay)
5. Then proceeds to actual scraping

TCG/OCG Filtering System

Understanding TCG vs OCG

TCG (Trading Card Game):

- Western market (North America, Europe, etc.)
- English, German, French, Spanish, Italian language cards
- Different card pool and release schedule
- Most Cardmarket users want TCG cards

OCG (Official Card Game):

- Japanese and Asian market
- Japanese and Korean language cards
- Different card pool, often ahead of TCG
- Less relevant for most Cardmarket users
- **Filtered out by this script**

Detection Logic

How OCG is identified:

```
def is_ocg_expansion(expansion_name):
    return 'OCG' in expansion_name
```

Detection criteria:

- Checks if string "OCG" appears anywhere in expansion name
- **Case-sensitive**: Only uppercase "OCG" is detected
- Examples that will be detected:
  - `"Booster Pack 1 (OCG)"`
  - `"OCG Exclusive Set"`
  - `"Structure Deck - OCG Edition"`
  - `"Limited Pack OCG"`

Why this works:

- Cardmarket consistently labels OCG expansions with "OCG" marker

- Always uppercase in expansion names
- Appears as suffix, prefix, or in parentheses
- Simple, reliable detection method

Filtering Process

During extraction:

```
is_ocg = is_ocg_expansion(expansion_name)
target_dict = ocg_expansions_dict if is_ocg else tcg_expansions_dict
```

Steps:

1. For each expansion option found
2. Check if name contains "OCG"
3. If OCG: Add to `ocg_expansions_dict`
4. If TCG: Add to `tcg_expansions_dict`
5. Track both lists separately

Final output:

- `tcg_expansions`: Saved to output file
- `ocg_expansions`: Excluded, but shown in console

Statistics Reporting

Console output shows:

```
✓ Found 1211 total expansions
  - TCG expansions: 987
  - OCG expansions (excluded): 224

Excluded OCG expansions (first 5):
  [ 4665] Yu-Gi-Oh! The Dark Side of Dimensions Movie Pack (OCG)
  [ 4787] Absolute Powerforce OCG
  ...
```

Information provided:

- Total expansions found (TCG + OCG)
- Count of TCG expansions (will be saved)
- Count of OCG expansions (excluded)
- Examples of excluded OCG expansions (first 5)
- Full list of TCG expansions to be saved

Benefits:

- User sees what is being excluded
- Can verify OCG detection is working correctly
- Transparent about filtering decisions
- Easy to spot issues if detection fails

Duplicate Prevention System

The Duplicate Problem

Discovery:

- Initial extraction yielded 2422 entries
- Validation revealed only 1211 unique IDs
- Each expansion appeared twice in the list

Root cause:

- Cardmarket may organize expansions in multiple optgroups
- Some expansions may appear in multiple categories
- HTML structure contains intentional duplicates for UX purposes
- Manual extraction included all occurrences

Dictionary-Based Solution

Implementation (per TCG/OCG category):

```
tcg_expansions_dict = {}
ocg_expansions_dict = {}
duplicate_count = 0

for option in options:
    expansion_id = int(value)
    is_ocg = is_ocg_expansion(expansion_name)
    target_dict = ocg_expansions_dict if is_ocg else tcg_expansions_dict

    if expansion_id in target_dict:
        duplicate_count += 1
        continue  # Skip duplicate

    target_dict[expansion_id] = {
        'expansion_id': expansion_id,
        'expansion_name': expansion_name
    }

tcg_expansions = list(tcg_expansions_dict.values())
ocg_expansions = list(ocg_expansions_dict.values())
```

Why this works:

- Dictionary keys are unique by definition
- Attempting to add duplicate key overwrites (we skip instead)
- First occurrence is preserved (per category)
- O(1) lookup time for duplicate checking
- No nested loops or complex logic required
- **Works independently for TCG and OCG**

First-Occurrence Strategy

Decision: Keep the first occurrence of each expansion ID (per category)

Rationale:

- First occurrence is typically the "main" category
- Subsequent occurrences are cross-references
- Maintains original dropdown order
- Simpler than comparing names for "best" version

Alternative considered:

- Could compare expansion names and keep longest/shortest
- Could merge names from duplicates
- Rejected as unnecessary complexity

Validation Layer

Double-check after extraction:

```
tcg_ids = [exp['expansion_id'] for exp in tcg_expansions]
unique_tcg_ids = set(tcg_ids)

if len(tcg_ids) != len(unique_tcg_ids):
    # Emergency deduplication
    seen = set()
    deduplicated = []
    for exp in tcg_expansions:
        if exp['expansion_id'] not in seen:
            seen.add(exp['expansion_id'])
            deduplicated.append(exp)
    tcg_expansions = deduplicated
```

Fail-safe approach:

- Assumes primary deduplication might fail
- Performs secondary check on final TCG list

- Uses set comparison for quick validation
- Implements emergency deduplication if needed
- Guarantees output is duplicate-free

When this triggers:

- If dictionary approach somehow fails
- If data structure changes unexpectedly
- Provides defensive programming safety net

Reporting

User feedback:

```
if duplicate_count > 0:
    print(f"⚠ Removed {duplicate_count} duplicate entries")

print(f"✓ Verified: All {len(tcg_expansions)} TCG expansions are unique")
```

Transparency:

- Informs user of duplicates found and removed
- Confirms final TCG list is clean
- Provides confidence in data quality
- Useful for debugging if issues arise

Usage Instructions

Quick Start

Step 1: Install dependencies

```
pip install cloudscraper beautifulsoup4
```

Step 2: Run the script

```
python cardmarket_expansion_list_scraper.py
```

Step 3: Check output

```
# View file size
ls -lh cardmarket_expansion_list.json

# Preview content (first 20 lines)
head -20 cardmarket_expansion_list.json
```

```
# Count TCG expansions
cat cardmarket_expansion_list.json | grep "expansion_id" | wc -l
```

Expected Console Output

```
======================================================================
 CARDMARKET EXPANSION LIST EXTRACTOR v1.3 (TCG-Only)
======================================================================

Creating scraper...
Warming up session...
Fetching expansion list from Cardmarket...
URL: https://www.cardmarket.com/en/YuGiOh/Products/Search?...

✓ Page loaded successfully (137222 bytes)

Extracting expansion list...
Found 1212 total options in dropdown
⚠ Removed 0 duplicate entries
✓ Found 1211 total expansions
  - TCG expansions: 987
  - OCG expansions (excluded): 224

Excluded OCG expansions (first 5):
  [ 4665] Yu-Gi-Oh! The Dark Side of Dimensions Movie Pack (OCG)
  [ 4787] Absolute Powerforce OCG
  [ 4796] Ancient Prophecy OCG
  [ 4760] Beginners Edition 1 2011
  [ 4756] Beginners Edition 2 2011
  ... and 219 more

TCG expansions to be saved:
First 5:
  [ 1651] 2-Player Starter Deck Yuya & Declan
  [ 5420] 2-Player Starter Set
  [ 1433] 2013 Zexal Collection Tin
  [ 1497] 2014 Mega-Tins
  [ 1498] 2014 Mega-Tins Mega-Pack
  ...
Last 5:
  [ 1674] Yugi's Legendary Decks
  [ 1442] Yugioh Filler Cards
  [ 1402] Yugioh Products
  [ 1414] ZEXAL Manga Promos
  [ 1492] ZEXAL World Duel Carnival Promos

✓ Verified: All 987 TCG expansions are unique

Saving TCG expansions to cardmarket_expansion_list.json...
✓ Saved 987 TCG expansions to cardmarket_expansion_list.json
  (Excluded 224 OCG expansions)

======================================================================
```

```
 COMPLETE
======================================================================
```

Integration with Card Scraper

Usage pattern:

```
# In cardmarket_card_list_scraper.py
with open('cardmarket_expansion_list.json', 'r', encoding='utf-8') as f:
    expansions = json.load(f)

# Now iterate through TCG expansions only
for expansion in expansions:
    expansion_id = expansion['expansion_id']
    expansion_name = expansion['expansion_name']
    # Scrape cards for this TCG expansion...
```

Workflow:

1. Run expansion list scraper (once) → Gets TCG expansions
2. Verify `cardmarket_expansion_list.json` was created
3. Run card list scraper (uses the TCG expansion list)
4. Re-run expansion scraper periodically to catch new TCG releases

Validation Commands

Check TCG expansion count:

```
python -c "
import json
with open('cardmarket_expansion_list.json') as f:
    data = json.load(f)
print(f'TCG expansions: {len(data)}')
"
```

Verify no OCG in output:

```
python -c "
import json
with open('cardmarket_expansion_list.json') as f:
    data = json.load(f)
ocg_found = [e for e in data if 'OCG' in e['expansion_name']]
print(f'OCG expansions in file: {len(ocg_found)}')
if ocg_found:
    for e in ocg_found:
        print(f\"  [{e['expansion_id']}] {e['expansion_name']}\")
else:
    print('✓ No OCG expansions found (correct!)')
"
```

Find specific TCG expansion:

```
cat cardmarket_expansion_list.json | python -c "
import json, sys
data = json.load(sys.stdin)
search = 'Mega-Tins'
results = [e for e in data if search in e['expansion_name']]
for r in results:
    print(f\"[{r['expansion_id']}] {r['expansion_name']}\")
"
```

Troubleshooting

Common Issues

1. No Expansions Extracted

Symptoms:

```
✗ No expansions found!
Debug: Saving HTML to debug.html for inspection
```

Causes:

- HTML structure changed on Cardmarket
- Select element not found
- All options filtered out

Solutions:

1. Check `debug.html` file created
2. Search for `<select` to find dropdowns
3. Verify `name="idExpansion"` attribute exists
4. Check if options have numeric values
5. Update selector in `extract_expansions()` if needed

2. All Expansions Marked as OCG

Symptoms:

```
✓ Found 1211 total expansions
  - TCG expansions: 0
  - OCG expansions (excluded): 1211
```

Causes:

- OCG detection logic malfunction
- All expansion names contain "OCG" (unlikely)

Solutions:

1. Check `is_ocg_expansion()` function
2. Verify case sensitivity ("OCG" vs "ocg")
3. Examine first 5 excluded expansions - do they actually contain "OCG"?
4. If not, bug in detection logic - check string matching

3. Too Many/Too Few OCG Exclusions

Expected: ~200-300 OCG expansions excluded

If too many (>400):

- Detection too broad, catching non-OCG expansions
- Check examples of excluded expansions
- Verify they actually contain "OCG"

If too few (<100):

- Detection too narrow, missing OCG expansions
- Some OCG sets might use different naming
- Check Cardmarket directly for OCG naming patterns

4. HTTP Errors

Symptoms:

```
✗ Error: HTTP 403
  Response: Forbidden
```

Causes:

- IP blocked by Cardmarket
- CloudScraper failed to bypass protection
- Network connectivity issues

Solutions:

1. Wait 1 hour and retry
2. Try from different network
3. Update CloudScraper: `pip install --upgrade cloudscraper`
4. Verify you can access the URL in browser manually

5. Connection Timeout

Symptoms:

```
✗ Error fetching page: HTTPSConnectionPool(...): Read timed out
```

Solutions:

1. Check internet connection
2. Increase timeout in `scraper.get()` call
3. Try again during off-peak hours
4. Check if Cardmarket.com is accessible

6. Import Errors

Symptoms:

```
ModuleNotFoundError: No module named 'cloudscraper'
```

Solution:

```
pip install cloudscraper beautifulsoup4
# Or with Python 3 explicitly:
python3 -m pip install cloudscraper beautifulsoup4
```

Data Quality Checks

After successful run, verify:

1. **File exists and has content:**

```
ls -lh cardmarket_expansion_list.json
# Should be ~60-80 KB for TCG-only
```

2. **Valid JSON:**

```
python -m json.tool cardmarket_expansion_list.json > /dev/null
# No output = valid JSON
```

3. **Expected TCG count:**

```
cat cardmarket_expansion_list.json | grep "expansion_id" | wc -l
# Should be ~900-1000 (TCG only)
```

4. **No OCG in output:**

```
grep -i "OCG" cardmarket_expansion_list.json
# Should return nothing (no matches)
```

5. **Sample entries look correct:**

```
head -30 cardmarket_expansion_list.json
# Check structure and verify no OCG names
```

Debugging Tips

Enable CloudScraper logging:

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

Print raw HTML snippet:

```
print(html[:1000])  # First 1000 characters
```

Inspect BeautifulSoup parsing:

```
print(soup.prettify()[:2000])  # First 2000 chars formatted
```

Check found elements:

```
select_element = soup.find('select', attrs={'name': 'idExpansion'})
print(f"Found: {select_element is not None}")
if select_element:
    options = select_element.find_all('option')
    print(f"Options count: {len(options)}")
    # Check for OCG
    ocg_count = sum(1 for opt in options if 'OCG' in opt.get_text())
    print(f"OCG options: {ocg_count}")
```

Maintenance

Update frequency:

- Run monthly to catch new TCG expansion releases
- Yu-Gi-Oh! releases ~4-6 new TCG sets per year
- OCG sets release more frequently but are filtered out
- More frequent updates not necessary

Version compatibility:

- Python 3.7+

- CloudScraper (any recent version)
- BeautifulSoup4 (any recent version)

Future-proofing:

- If Cardmarket changes HTML structure, update `extract_expansions()`
- If URL changes, update `SEARCH_URL` constant
- If OCG naming convention changes, update `is_ocg_expansion()`
- If anti-bot protection increases, may need Selenium instead

Technical Notes

Why CloudScraper vs Requests

Requests library limitation:

```
# This would fail:
response = requests.get(SEARCH_URL)
# Cardmarket uses Cloudflare protection
```

CloudScraper advantage:

- Automatically solves JavaScript challenges
- Handles Cloudflare protection
- Transparent - same API as requests
- No browser automation needed

Performance Characteristics

Speed:

- Warmup request: ~1-2 seconds
- Main request: ~1-2 seconds
- Parsing + OCG filtering: <0.1 seconds
- Total: ~5-10 seconds

Memory:

- HTML content: ~140 KB
- Parsed structure: ~500 KB RAM
- Output JSON: ~60-80 KB disk (TCG-only)
- Peak RAM usage: <50 MB

Network:

- Total download: ~150 KB
- Total upload: ~5 KB (headers)
- Requests: 2
- No pagination required

Error Recovery

Automatic:

- CloudScraper retries failed requests
- BeautifulSoup handles malformed HTML
- Dictionary deduplication is fail-safe

Manual recovery needed:

- HTTP 403 (wait and retry)
- Connection timeout (check network)
- Module import errors (install dependencies)

Alternative Approaches Considered

Selenium + browser automation:

- **Pros**: Handles all JavaScript, very robust
- **Cons**: Slow, requires ChromeDriver, heavy
- **Decision**: CloudScraper sufficient for this use case

API access:

- **Pros**: Official, stable, fast
- **Cons**: Cardmarket has no public API for expansion lists
- **Decision**: Not available

Manual maintenance:

- **Pros**: 100% accurate, no scraping
- **Cons**: Tedious, error-prone, outdated quickly, can't distinguish TCG/OCG easily
- **Decision**: Automation preferred

Include both TCG and OCG with flag:

- **Pros**: More complete data
- **Cons**: Downstream scripts must filter, larger file, most users don't need OCG
- **Decision**: TCG-only output is cleaner

Best Practices

Before Running

1. ✓ Ensure stable internet connection
2. ✓ Install dependencies with pip
3. ✓ Verify Python version (3.7+)
4. ✓ Check Cardmarket.com is accessible in browser

After Running

1. ✓ Verify output file size (~60-80 KB for TCG-only)
2. ✓ Check JSON validity
3. ✓ Confirm TCG expansion count (~900-1000)
4. ✓ Validate no OCG expansions in output
5. ✓ Review excluded OCG list in console (should show ~200-300)
6. ✓ Backup output file before modifications

Regular Maintenance

1. ✓ Update monthly for new TCG expansions
2. ✓ Keep CloudScraper library updated
3. ✓ Archive previous versions for comparison
4. ✓ Test after Cardmarket UI updates
5. ✓ Verify OCG detection still works (check console output)

Conclusion

The Cardmarket Expansion List Extractor is a focused, efficient tool for obtaining **TCG-only** Yu-Gi-Oh! expansion metadata from Cardmarket.com. Its simple design, robust duplicate prevention, **OCG filtering**, and reliable extraction logic make it an essential component of the Cardmarket scraping workflow.

Key strengths:

- Single-purpose, easy to understand
- **Automatic TCG/OCG separation**
- Duplicate-free output guaranteed
- Fast execution (~5-10 seconds)
- Minimal dependencies
- Robust error handling

- **Informative console output** showing what's excluded

Limitations:

- Requires working CloudScraper
- Subject to Cardmarket's HTML structure
- No pagination (not needed for expansion list)
- English language only
- **OCG detection based on naming convention** (relies on "OCG" marker)

Use cases:

- Generate input for card list scraper (TCG cards only)
- Track new TCG expansion releases
- Market analysis of TCG vs OCG availability
- Automated data collection pipelines

For questions, issues, or updates, refer to this documentation and the inline code comments.